JavaScript

JavaScript es un lenguaje de programación utilizado por los desarrolladores para añadir funcionalidad a las páginas web. Este dinamismo puede estar en la parte del cliente (front) o en la parte del servidor web (back)

JS en el cliente: el código JavaScript incorpora diferentes funciones para gestionar el navegador y el modelo de objetos del documento (DOM). Estas funciones realizan cambios en HTML y CSS. El código es interpretado por el motor de JavaScript incorporado en el navegador (motor V8 de Google, por ejemplo). Además, el código solo puede acceder a aquellos recursos que el navegador le permita.

JS en el servidor: una función de JavaScript del lado del servidor puede acceder a la base de datos o realizar diferentes operaciones de la lógica de la aplicación. En este caso se requiere la instalación en el servidor de un intérprete JS, como puede ser Node.js, React o Vue.js

JavaScript en el código HTML ➡ etiqueta <script>

```
<!DOCTYPE html>
<html lang="es">
<head>
  ...
  <script src=prueba.js ></script>
  <script>alert("Hola")</script>
</head>
<body>
  <script>
   document.write("<p> Añadido con js </p>");
  </script>
  <p onclick="console.log('Adiós')"> Pulsa aquí</p>
```

Se pueden utilizar tantas etiquetas <script> como se necesiten

Recurso externo: el atributo src especifica la URI del script externo

Directamente: el código JavaScript se incluye en la etiqueta <script>. Esta etiqueta puede aparecer en cualquier lugar del documento HTML (head, body o ambos)

Directamente: dentro del atributo de un elemento HTML en respuesta a un evento determinado

JavaScript en el código HTML: etiqueta <script>

```
<script async src="js/jquery.js"></script>
<script async src="js/script2.js"></script>
<script async src="js/script3.js"></script>
```

Tanto ***async*** como ***defer*** indican al navegador que descargue los scripts en un hilo separado, mientras el resto de la página (DOM) se descarga.

La carga de la página no se bloquea mientras se obtienen los scripts

```
<script defer src="js/jquery.js"></script>
<script defer src="js/script2.js"></script>
<script defer src="js/script3.js"></script>
```

*Los scripts con el atributo **async** se ejecutarán tan pronto como se complete su descarga. Esto bloquea la página y no garantiza un orden de ejecución específico.*

*Usa **async** si tus scripts deben ejecutarse de inmediato y no tienen dependencias.*

*Los scripts con el atributo **defer** se cargarán en el orden en que están en la página y solo se ejecutarán cuando todo haya terminado de cargarse.*

*Usa **defer** si tus scripts necesitan que se analice el contenido o dependan de otros scripts o de que el DOM esté en su lugar. Pon los elementos <script> correspondientes en el orden en que deseas que el navegador los ejecute.*

Consideracions generales en JavaScript

- Las sentencias JavaScript finalizan con **;**

  Aunque no se recomienda, se permiten varias sentencias en la misma línea.

- JavaScript es Case-Sensitive.

- Son comentarios:
  - el código después de //
  - el código entre /* y */

- Las declaraciones de JavaScript se agrupan en bloques de código, dentro de llaves {...}

- Para nombres de variables o funciones se utilizan letras mayúsculas y minúsculas, dígitos, el signo dólar ($) y el carácter subrayado (_)

  NO SE PERMITEN GUIONES. Un identificador ***no*** puede empezar por dígito.

- Las funciones se declaran con la palabra reservada ***function***

  function nombre_function ***() { ... }***

Aritméticos

- **+** *suma*
- **-** *resta*
- ***** *multiplicación*
- **/** *división*
- **%** *módulo*
- ****** exponente
- **++** *incremento*
  - *++n : incrementa y utiliza*
  - *n++: utiliza e incrementa*
- **--** *decremento*
  - *--n : decrementa y utiliza*
  - *n--: utiliza y decrementa*

A nivel de Bit

- **&** *and* a nivel de bits
- **|** *or* a nivel de bits
- **^** *xor* a nivel de bits
- **~** *not* a nivel de bits
- **<<** desplazamiento de bits a la izquierda
- **>>** desplazamiento de bits a la derecha

Lógicos

- **&&** *operador AND*
- **||** *operador OR*
- **!** *operador NOT*

Asignación

- **x = y**
- **x += y**
- **x -= y**
- **x *= y**
- **x /= y**
- **x %= y**
- **x **= y**

Comparación

- **==** *igual*
- **===** *igual valor y tipo*
- **!=** *distinto*
- **!==** *distinto en valor y tipo*
- **>** mayor
- **>=** mayor o igual
- **<** menor
- **<=** menor o igual
- **?** operador ternario

SENTENCIAS CONDICIONALES

```
if (condition) {
    block of code
}
```

```
if (condition) {
  code condition - true
} else {
  code condition - false
}
```

```
if (condition_1) {
  code
} else if (condition_2) {
  code
} else if (condition_3) {
  code
} else {
  code
}
```

```
switch (expression) {
    case value_1:
        code
        break;
    case value_2:
        code
        break;
    default:
        code
}
```

Operador condicional

condición a verificar

Valor a devolver si condición es falsa

```
precio > 1000 ? "caro" : "barato"
```

Valor a devolver si la condición es verdadera

!!! ***pass-through***:
si no se añaden los break, se ejecuta todo el código desde el primer caso que coincide hasta el final de la sentencia switch.

BUCLES

```
while (condition) {
    block of code
}
```

```
do {
    code block
} while (condition);
```

```
let i = 0;
while (i < 10) {
    console.log(i);
    i++;
}
```

```
for (let i = 0; i < 10; i++) {
    console.log(i);
}
```

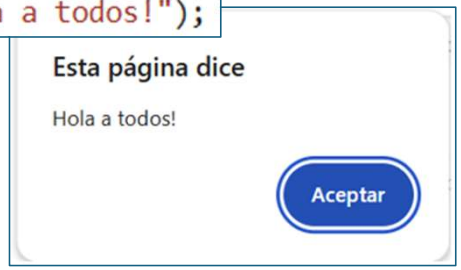
```
for (initialization; condition; increment) {
    block of code
}
```

```
for (variable of array) {
    block of code
}
// variable itera en los valores del array
```

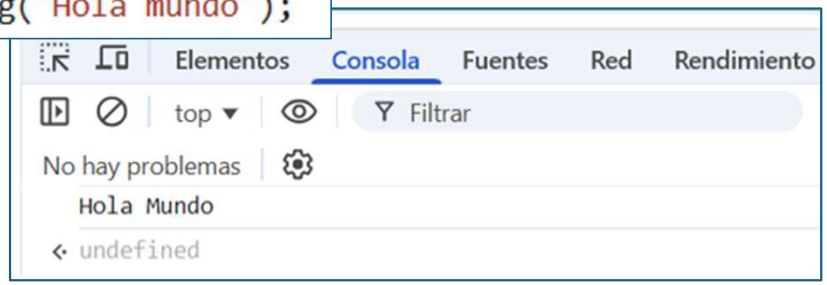
```
for (key in object) {
    block of code
}
// key itera en las claves del objeto
```

```
window.alert("Hola a todos!");
```

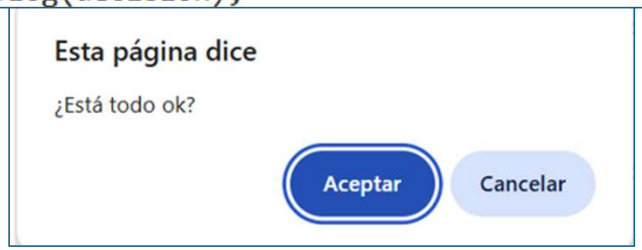
```
alert("Hola a todos!");
```




```
console.log("Hola mundo");
```




```
let decision = window.confirm("¿Está todo ok?");
console.log(decision);
```




```
let nombre = window.prompt("Introduce tu nombre", "Pepito");
nombre ? nombre : "Anonimo"
```

TIPOS DE DATOS

— **PRIMITIVOS**

- Boolean (true, false)
- Number (enteros o reales)
- BigInt
- String
- Symbol
- undefined

— **COMPLEJOS**

- Object
- Arrays

Number

```
a = 10;    // decimal
b = 0x10;  // hexadecimal
c = 0o10;  // octal
d = 0b10;  // binary
x = 9e3;   // 9000
y = 12e-5;// 0.00012
```

Valores especiales: ***Infinity –Infinity NaN***
NaN notificación de que no se pudo realizar alguna acción matemática porque el argumento no es un número o no se puede convertir a un número

Number.isFinite(x)
Permite comprobar si x es un número válido.

BigInt

```
big = 1234567890000000000n;
big2 = 1n;
big3 = 1000n + 20;
 Uncaught TypeError:
 Cannot mix BigInt and other types
```

null: valor empleado para marcar que una variable de tipo complejo no contiene ningún valor significativo.
typeof null -> Object

INTERPOLACIÓN DE CADENAS

```
nombre = "Amalia";
frase =` Mi nombre es ${nombre}.`;
```

MÉTODOS DE CADENAS

```
    str = "java script language";
str.length  -> 20
str.charAt(0)  -> 'j'
str.slice(0, 4)   -> 'java'
str.split(' ')  -> ['java', 'script', 'language']

'10.16.1.1'.split('.') ->  ['10','16','1','1']
```

Conversión de datos mediante las funciones constructoras

```
Number(42)       -> 42
Number("11")     -> 11
Number("0x11")   -> 17
Number("0o11")   -> 9
Number("0b11")   -> 3
Number("12e3")   -> 12000

Number("text")   -> NaN
Number(true)     -> 1
Number(false)    -> 0

Number(undefined)  -> NaN
Number("Infinity") -> Infinity
Number(null)       -> 0
```

```
BigInt(11)  ->  11n
BigInt(0x11) -> 17n
BigInt(11e2) -> 1100n

BigInt(true) -> 1n

BigInt("11")  -> 11n
BigInt("0x11") ->7n

BigInt(null) -> Uncaught TypeError: Cannot convert  ...
BigInt(undefined) -> Uncaught TypeError: Cannot convert  ...
BigInt(NaN)  ->  Uncaught RangeError: The number NaN ...
```

```
Boolean(true)    ->  true
Boolean(42)      ->  true
Boolean(0)       ->  false
Boolean("text") ->  true

Boolean("")      ->  false
Boolean(NaN)     ->  false
Boolean(null)    ->  false
Boolean(undefined) ->  false
```

```
num = 42;
String(num)  -> "42"
String(8)    -> "8"
String(true) -> "true"
String(Infinity) -> "Infinity"
```

- JS es un lenguaje de tipado ***débil***: una variable ***cambia de tipo*** según su contenido.
- Las variables se crean cuando se ***declaran***. Para declarar una variable se utilizan las palabras reservadas: ***let***, ***const***, y ***var***.
- El valor asignado a una variable cuando se declara es **"undefined". Antes de poder utilizarla hay que inicializarla.**

"***use strict";*** obliga al intérprete a comportarse de acuerdo con los estándares modernos de JavaScript.

Usar una variable sin su declaración previa se trata como un error.

- ***const*** - declara variables que no permiten modificar su valor o su tipo. Realmente son constantes.

  Las variables const **deben inicializarse en la declaración.**
- ***var*** - Las variables declaradas con var tienen alcance ***global***, ***excepto*** si se declaran en una ***función***, en cuyo caso el alcance es **local** a la función.

  Si se vuelve a declarar una variable declarada con var, conserva el valor anterior (no "undefined")
- ***let, const*** **–** las variables así declaradas **dentro de un bloque**, serán ***locales*** a ese **bloque** (visibles también en bloques que se puedan anidar en él)

  let , const -> *No permite* declarar otra variable con el mismo nombre ***EN EL MISMO ÁMBITO*** evitando redeclaraciones accidentales

TIPOS DE DATOS

— **COMPLEJOS**
- Object
- Array

```
user1 = {
   name:  "Calvin",
   surname:  "Hart",
   age:  66,
   email:  "CalvinHart@telefonica.net"
};
```

```
user1.name -> "Calvin"
user1.surname -> "Hart"
```

```
// Añadir un nuevo campo al objeto
user1.telephone = 555 555 555;
```

```
// Borrar un campo del objeto
delete user1.age;
```

```
days = ["Mon", "Tue", "Wed"];
days[0]  =  "Monday";
days[6]  =  "Sunday";
  days[5] -> undefined;
```

```
days.length -> 7
```

```
typeof days -> Object
days instanceof Array -> true
```

```
values = [["Test",  7,  12.3,  false],
          ["Prueba",  true, 9.5]];
```

```
values[0][2] -> 12.3
values[1][3] -> undefined
```

```
values.length -> 2
```

PROGRAMACIÓN DE INTELIGENCIA ARTIFICIAL

JavaScript

TIPOS DE DATOS

ARRAYS

CREACIÓN DE UN ARRAY

```
let array = new Array();
let array = Array();
let array = [];
```

```
let array = new Array(3);
- - > Crea un array de tres posiciones sin inicializar los valores
```

```
let array = new Array(22,20,18,16);
let array = Array(22,20,18,16);
let array = [22,20,18,16];
```

RECORRIDO DE UN ARRAY

```
for (let i=0; i < array.length; i++) {... array[i] ...}

for (let i in array) {   ... array[i] ...}
```

METODOS DE ARRAYS

```
days.indexOf("Mon")
// Devuelve la posicion del elemento

days.push("Sat")
//Inserta en la última posición

days.unshift("Sun")
//Inserta en la primera posición

days.pop()
// Elimina y devuelve el último elemento

days.shift()
// Elimina y devuelve primer elemento

days.reverse()
// Invierte la matriz
```

```
days = ["Mon", "Tue", "Wed","Thu","Fri"]

days.slice(2)
 -> ["Mon","Tue"]

days.slice(1,3)
-> ["Tue", "Wed"]

days.slice(0,-1)
-> ["Mon", "Tue", "Wed","Thu"]

days.slice(-2)
-> ["Thu","Fri"]
```

```
months = ["Jan","Feb"];

months.concat(days)
 -> ["Jan","Feb","Mon","Tue",...]
```